

DSA

1. Sorting Algorithms

1. Bubble Sort

Idea

Compare adjacent elements and swap if they are in the wrong order. Largest element "bubbles" to the end in each pass.

Example

Array: [5, 3, 8, 2]

Pass 1:

- $5 > 3 \rightarrow \text{Swap} \rightarrow [3, 5, 8, 2]$
- $5 < 8 \rightarrow \text{No swap}$
- $8 > 2 \rightarrow \text{Swap} \rightarrow [3, 5, 2, 8]$

Pass 2:

- $3 < 5$
- $5 > 2 \rightarrow \text{Swap} \rightarrow [3, 2, 5, 8]$

Pass 3:

- $3 > 2 \rightarrow \text{Swap} \rightarrow [2, 3, 5, 8]$

Time Complexity

Case	Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

2. Insertion Sort

Idea

Insert each element into its correct position in the already sorted part.

Example

Array: [5, 3, 8, 2]

- Insert 3 before 5 → [3, 5, 8, 2]
- 8 already correct → [3, 5, 8, 2]
- Insert 2 at beginning → [2, 3, 5, 8]

Time Complexity

Case	Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

3. Quick Sort

Idea

Choose a pivot and partition elements into:

- Smaller than pivot
- Greater than pivot

Then recursively sort both parts.

Example

Array: [7, 2, 1, 6, 8, 5, 3, 4]

Pivot = 4

[2, 1, 3] 4 [7, 6, 8, 5]

Recursively sort both sides.

Time Complexity

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

Worst case occurs when pivot is always smallest/largest.

Advantages

- Fastest practical sorting
- Cache friendly
- In-place

Disadvantages

- Worst case $O(n^2)$
- Not stable

4. Merge Sort

Idea

Divide array into halves, sort each half, then merge.

Example

Array:

```
[8,3,5,4]
```

Split:

```
[8,3] [5,4]
```

Further split:

```
[8][3][5][4]
```

Merge:

```
[3,8]  
[4,5]
```

Final:

```
[3,4,5,8]
```

Whenever there is an ODD size of the array, the splitting of the array is left part is less no. of elements and right more number of element

eg. size = 5 [1, 2, 3, 4, 5] → [1, 2] [3, 4, 5]

Time Complexity

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Advantages

- Stable
- Guaranteed $O(n \log n)$

Disadvantages

- Extra memory $O(n)$

2. Arrays

What is an Array?

Collection of elements stored in contiguous memory locations.

Example:

```
int[] arr= {10,20,30,40};
```

Memory:

```
1000 -> 10  
1004 -> 20  
1008 -> 30  
1012 -> 40
```

Array Traversal

Visiting each element once.

```
for(int i=0;i<arr.length;i++)  
{  
    System.out.println(arr[i]);  
}
```

Complexity

$O(n)$

Searching in Arrays

Linear Search

Check one by one.

Example:

Find 30

```
10 20 30 40
      ^
```

Complexity:

$O(n)$

Binary Search

Works only on sorted arrays.

Example:

```
10 20 30 40 50
```

Search 30

Middle = 30 found immediately.

Complexity:

$O(\log n)$

Time Complexity of Array Operations

Operation	Complexity
Access	$O(1)$
Search	$O(n)$
Insert End	$O(1)$
Insert Middle	$O(n)$
Delete	$O(n)$
Traverse	$O(n)$

When to Use Arrays

Use when:

- Fixed size data
- Frequent indexing
- Need $O(1)$ access

Examples:

- Student marks
- Monthly sales
- Sensor readings

Alternatives

- Linked List
- ArrayList
- HashMap

3. Linked Lists

Singly Linked List

Node contains:

```
Data + Next Pointer
```

Example:

```
10 → 20 → 30 → NULL
```

Complexities

Operation	Complexity
Search	$O(n)$
Insert Beginning	$O(1)$

Operation	Complexity
Insert End	$O(n)$
Delete	$O(n)$
Traverse	$O(n)$

Use Cases

- Dynamic memory allocation
- Undo operations

Circular Singly Linked List

Last node points to first node.

```

10 → 20 → 30
↑         ↓
← ← ← ← ←

```

Advantages

- No NULL pointer
- Useful in round-robin scheduling

Use Cases

- CPU scheduling
- Multiplayer game turns

Doubly Linked List

Each node contains:

```
Prev | Data | Next
```

Example:

```
NULL ← 10 ⇌ 20 ⇌ 30 → NULL
```

Advantages

- Forward and backward traversal

Disadvantages

- Extra memory for prev pointer

Use Cases

- Browser history
 - Music playlist
-

Circular Doubly Linked List

```
10 ⇌ 20 ⇌ 30
↑           ↓
← ← ← ← ← ←
```

Advantages

- Traverse both directions endlessly

Use Cases

- Media players
 - Operating systems
-

Linked List Operations

Search

Traverse until element found.

Complexity:

$O(n)$

Insert Beginning

```
20→30
```

```
Insert 10
```

```
10→20→30
```

Complexity:

$O(1)$

Insert End

```
10→20
```

```
Insert 30
```

```
10→20→30
```

Complexity:

$O(n)$

Delete

```
10→20→30
```

```
Delete 20
```

```
10→30
```

Complexity:

$O(n)$

Array vs Linked List

Feature	Array	Linked List
Memory	Contiguous	Dynamic
Access	$O(1)$	$O(n)$
Insert	$O(n)$	$O(1)$ at head
Delete	$O(n)$	$O(1)$ at head
Cache Friendly	Yes	No

4. Searching Algorithms

Linear Search

Idea

Check every element one by one.

Example

Search 40

```
10 20 30 40 50
           ↑
```

Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

Advantages

- Works on unsorted data

- Easy implementation

Disadvantages

- Slow for large data

Use Cases

- Small datasets
- Unsorted arrays

Alternative

- Binary Search
-

Binary Search

Idea

Repeatedly divide search space into half.

Example

Array:

```
10 20 30 40 50 60 70
```

Search 60

Step 1:

Middle = 40

```
60 > 40
```

Search right half

Step 2:

Middle = 60

Found

Complexity

Case	Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

Advantages

- Extremely fast
- Suitable for large data

Disadvantages

- Array must be sorted

Use Cases

- Dictionary search
- Database indexing
- Searching products

Alternative

- Linear Search
-

Interview Summary (Most Important)

Topic	Key Point
Bubble Sort	Adjacent swapping
Insertion Sort	Insert in sorted portion
Heap Sort	Heap-based sorting

Topic	Key Point
Quick Sort	Pivot-based partitioning
Merge Sort	Divide and merge
Array Access	$O(1)$
Linked List Search	$O(n)$
Linear Search	$O(n)$
Binary Search	$O(\log n)$, sorted data required
Array vs Linked List	Fast access vs dynamic insertion